main.py

```
import tkinter as tk
import time
import random
#i looked at youtube videos on tutorials on using tkinter and other features, but I did not copy down any code and only used the methods and skills I learned from those videos in my coding.
#I did ask friends to help test my game, debug and etc. They did not write any code for me, and only gave me suggestions for how I can improve the code. I was present when the code was tested, and I used trial and error when figuring out solutions.
root = tk.Tk()
root.configure(bg="grey")
root.title("A Mindsweeper Game ")
root.geometry('1470x750')
root.eval('tk::PlaceWindow . center')
#Makes it so user cannot change the size of the window
root.resizable(False, False)
#for settings
ROWS = 9
COLS = 9
MINES = 10
#colors
#I didn't want basic colors, so I went to a color picker and got the hex number for the colors I think looked the coolest.
COLORS = {1:"#1565C0", 2:"#2E7D32", 3: "#C62828", 4: "#4527A0", 5: "#6D4C41", 6: "#00838F", 7: "#AD1457", 8: "#424242"
}

#a list of the directions around the cell/square. This helps optimize to check how many bombs are around a square the player clicks.
#The pairs are formatted like (row change, column change)
#e.g. (-1, 0) means go up 1 row and stay in the same column
#I made it look like a square so i can visualize it better
Directions = [(-1, -1),(-1, 0), (-1, 1),
              (0, -1),          (0, 1),
              (1, -1),  (1, 0), (1, 1)]

#the 3 different difficulties as a dictionary,
#The dictionary storing the settings for each difficulty.
#Each difficulty has its own number of rows, columns and mines.
Difficulty_settings = {
  "Easy": {"rows": 9, "cols": 9, "mines":10},
  "Medium": {"rows": 12, "cols": 12, "mines": 35},
  "Hard": {"rows": 12, "cols": 12, "mines": 99},
}

#board is a 2D list (a list of lists) that stores data for every cell
#buttons is also a 2D list that stores the tkinter button in every cell
```

```
board = []
buttons = []

#Variables that track the current state of the game
#turns true when game is won or lost
GameOver = False
#turns true after the player makes their first move
GameStarted = False
#players get as many flags as there are mines
FlagLeft = MINES
StartTime = 0
timerID = None

#The main screens for my game. Each screen is a frame. I use center frame inside every screen to keep everything centred. This was a challenge I faced, but after some water and hard thinking, I managed to solve it.
screen_main = tk.Frame(root, bg="grey")
main_center = tk.Frame(screen_main, bg = "grey")
main_center.place(relx = 0.5, rely = 0.5, anchor = "center")

screen_info = tk.Frame(root, bg="grey")
info_center = tk.Frame(screen_info, bg = "grey")
info_center.place(relx = 0.5, rely = 0.5, anchor = "center")

screen_options = tk.Frame(root, bg="grey")
options_center = tk.Frame(screen_options, bg = "grey")
options_center.place(relx = 0.5, rely = 0.5, anchor = "center")

screen_game = tk.Frame(root, bg="grey")
game_center = tk.Frame(screen_game, bg = "grey")
game_center.place(relx = 0.5, rely = 0.5, anchor = "center")

#this loops over every screen to place them identically
screens = [screen_main, screen_game, screen_info, screen_options]
for screen in screens:
  screen.place(relx = 0, rely = 0, relwidth = 1, relheight = 1)
def show_screen(screen):
  #positions the screen so it can be seen over other windows. e.g. if you opened options and then you opened info, this would place the info window above the options window.
  screen.tkraise()
#this is my main menu screen
tk.Label(main_center, text="Retro Mindsweeper",
         font=("Arial", 70, "bold"), bg="grey", fg="white").pack()
tk.Label(main_center, text="Now in Python!",
        font=("Arial", 50, "bold"), bg="grey", fg="blue").pack()
#The buttons for players to select their gamemode
#This is my list where each item is a pair. The button text and what it does when its clicked.
#Instead of writing 4 different buttons, I loop through this list and create them the
```

```
same way, just more efficiently.
menu_buttons = [
  ("Play!", lambda: start_game()),
  ("Info", lambda: show_screen(screen_info)),
  ("Options", lambda: show_screen(screen_options)),
  ("Exit", root.destroy),
]
#This is the loop that goes through each pair in menu_buttons
#text is the label on the button, and the action is what happens when you click it
for text, action in menu_buttons:
  tk.Button(
    main_center, text=text, command=action,
    font=("Arial", 18, "bold"),
    fg = "white", bg = "grey"
  ).pack()

#My info screen
#this is the title for the info screen
tk.Label(info_center, text="How to Play Minesweeper?",
         font=("Arial", 40, "bold"), fg = "white", bg = "grey").pack()
#each label is a seperate sentence that explains how the game is played
tk.Label(info_center, text="Left-Click to reveal a cell!",
         font=("Arial", 15), fg = "white", bg = "grey").pack()
tk.Label(info_center, text="Right-Click to place or remove a flag!",
         font=("Arial", 15), fg = "white", bg = "grey").pack()
tk.Label(info_center, text="Numbers on the cells show how many bombs touch that cell.",
         font=("Arial", 15), fg = "white", bg = "grey").pack()
tk.Label(info_center, text="Click the   , LOSS or WIN to reset the board!", font =
("Arial", 15), fg = "white", bg = "grey").pack()
tk.Label(info_center, text="Avoid all the BOMBS to WIN!   ", font=("Arial", 15), fg =
"white", bg = "grey").pack()
tk.Button(info_center, text="BACK",
          #it's show_screen(screen_main) because I need to raise the outer screen and
not the inner one.
          command=lambda: show_screen(screen_main), font = ("Arial", 15), fg = "white",
bg = "grey").pack()

#My options screen

tk.Label(options_center, text="Options", font=("Arial", 35, "bold"), fg ="white", bg =
"grey").pack()

tk.Label(options_center, text="Choose your Difficulty", font=("Arial", 20), fg
="white", bg = "grey").pack()
#StringVar stores which difficulty the player selected. It starts on easy by default.
difficulty_choose = tk.StringVar(value="Easy")
#This loop creates a radio button for each difficulty in the difficulty_settings. Radio
buttons only let the player pick one option at a time. This is good for difficulty
selection, as they can't pick more than 1.
for choice in Difficulty_settings:
```

```
    tk.Radiobutton(
128   options_center, text=choice,
129   variable = difficulty_choose, value = choice,
130   font = ("Times New Roman", 15),
131   fg = "white", bg = "grey",
132   selectcolor = "#57cfb9",
133   activebackground = "grey", activeforeground = "white"
134   ).pack()
135
136 def apply_selected_options():
137   #global means the actual rows columns and mines variables are changing.
138   global ROWS, COLS, MINES
139   #looks at which difficulty the player picked
140   config = Difficulty_settings[difficulty_choose.get()]
141   #updates the rows cols and mines to match the chosen difficulty
142   ROWS = config["rows"]
143   COLS = config["cols"]
144   MINES = config["mines"]
145   #reset the game with the new settings and go back to the main menu
146   new_game()
147   show_screen(screen_main)
148 #This button applies and saves the difficulty and goes back to the menu.
149 tk.Button(options_center, text="Apply!",
150          command = apply_selected_options,
151          font=("Arial", 14, "bold"),
152          fg="white", bg="#25852a").pack()
153
154 #Back button goes back without saving the changes
155 tk.Button(options_center, text="Back",
156   command=lambda: show_screen(screen_main), font = ("Times New Roman", 20), bg =
    "#9716a6", fg = "white").pack()
157
158 #This is the main game screen
159 #this frame holds the top bar consisting of menu button, flag count, reset button and
    the timer.
160 game_start = tk.Frame(game_center, bg = "grey")
161 game_start.pack()
162 #This button takes the player back to the main menu
163 tk.Button(game_start, text="Menu",
164           command = lambda: show_screen(screen_main),
165           font=("Arial", 15), bg="grey", fg="white").pack()
166
167 #shows the flags left the player can still place.
168 label_mines = tk.Label(game_start, text="Flags: " + str(MINES), font=("Times New
    Roman", 15, "bold"), bg="grey", fg="red", width = 10, anchor = "w")
169 #anchor = w just positions it to the left. w means west, which is towards the left.
170 label_mines.pack(side="left")
171 #reset button shows a smilely face when playing. it changes to either win or lose after
    the game ends
172 button_reset = tk.Button(game_start, text="  ", font = ("Times New Roman", 14), relief =
```

```
    "groove", command = lambda: new_game())
173 #Makes the text shift to the left.
174 button_reset.pack(side = "left", expand = True)
175 #shows the amount of time passed since the first click
176 label_timer = tk.Label(game_start, text = "Time: 0",
177 font =("Arial", 15, "bold"), bg = "grey", fg = "white", width = 10, anchor = "e")
178
179 label_timer.pack(side = "right")
180 #This frame is where the cell buttons get placed in a grid
181 board_frame = tk.Frame(game_center, bg = "grey")
182 board_frame.pack()
183 #This label displays the win/lose message after the game
184 label_msg = tk.Label(game_center, text = "", font = ("Times New Roman", 15), bg =
    "grey", fg = "white")
185 label_msg.pack()
186
187 #This is where the game logic is.
188 #This functions builds the board
189 def board_building():
190   #modify the actual board list
191   global board
192   board = []
193   #this is a nested loop (loop in a loop).
194   #The outer loop goes through each row, the inner loop goes through each column
195   for r in range(ROWS):
196     row = []
197     for c in range(COLS):
198       #Each cell is a dictionary that stores 4 things about that square.
199       row.append({
200         # is there a mine?
201         "mine": False,
202         # has the player clicked on the square?
203         "revealed": False,
204         #is there a flag here?
205         "flagged": False,
206         #How many mines are around the cell? (out of 8 surrounding cells.)
207         "count": 0
208       })
209     board.append(row)
210
211 def count_neighbours():
212   #This goes through every cell on the board
213   for r in range(ROWS):
214     for c in range(COLS):
215       #reset the count to 0 before checking this cell
216       board[r][c]["count"] = 0
217       #Loop through the 8 directions using the Directions list
218       for change_row, change_column in Directions:
219         neighbour_row = r + change_row
220         neighbour_column = c + change_column
```

```
          #makes sure the neighbour cell is actually on the board.
          if 0 <= neighbour_row < ROWS and 0 <= neighbour_column < COLS:
            if board[neighbour_row][neighbour_column]["mine"]:
              #if the neighbour is a mine, add 1 to the cell's count
              board[r][c]["count"] += 1

def reveal_cell(r,c):
  #These are the base cases. Conditions that stop the function from going on further
    #out of bounds vertically
  if r < 0 or r >= ROWS: return
    #horizontally out of bounds
  elif c < 0 or c>= COLS: return
    #already covered, so skip
  elif board[r][c]["revealed"]: return
    #already flagged, skip
  elif board[r][c]["flagged"]: return
    #never automatically reveal a mine
  elif board[r][c]["mine"]: return
    #mark as revealed and update how it looks on the screen
  board[r][c]["revealed"] = True
  draw_cell(r,c)
  #If the cell has 0 mines nearby, automatically reveal all the 8 neighbouring cells too
  #This is called recursion, where the function calls itself on each neighbour. Those
neighbours call it on their neighbours, and so on.
  if board[r][c]["count"] == 0:
    for change_row, change_column in Directions:
      reveal_cell(r + change_row, c + change_column)
#this function will basically draw the cells/squares to be more compatible with player
interactions
def draw_cell(r,c):
  #get data for this cell and its button widget
  cell = board[r][c]
  button = buttons[r][c]
  if cell["flagged"]:
    #F means flagged, with a yellow background so it looks good and stands out.
    button.config(text="F", bg="#ffe270", state = "normal", relief = "raised")
  #state determines if the button can be clicked or not. It's normal right now, so it
can be clicked
#relief is like the 3D appearance of the button. Raised means it looks raised and not
clicked.
  #cell hasn't been clicked yet, show it as a blank grey button.)
  elif not cell["revealed"]:
    button.config(text="", bg="grey", state="normal", relief = "raised")
  elif cell["mine"]:
    #X means bomb
    button.config(text="X", bg = "#c72c2c", state="disabled", relief = "sunken")
#here, state is disabled, meaning it can't be clicked. Relief is also sunken, so the
button/square/cell looks pressed down.
  elif cell["count"] > 0:
    button.config(text=str(cell["count"]), bg = "grey", state="disabled", relief =
```

```
"sunken", disabledforeground = COLORS[cell["count"]])
266       #disabledforeground changes the color depending on the number of mines around the square. It uses the colors dictionary
267     else:
268       button.config(text="", bg="grey", state = "disabled", relief = "sunken")
269   #left click
270
271   def left_click(r,c):
272     global GameOver, GameStarted, StartTime
273     #if the game is over, do nothing
274     if GameOver: return
275       #if the cell is uncovered, do nothing
276     if board[r][c]["revealed"]: return
277       #if the cell is flagged, do nothing
278     if board[r][c]["flagged"]: return
279       #on the very first click, start the timer and place the mines. Since the mines are placed here and after the first click (not the start), the first click is always safe. I implemented this after some buddies tested my code and died on the first attempt.
280     if not GameStarted:
281       GameStarted = True
282       StartTime = time.time()
283   #place mines but skip the cell the player just clicked
284       place_mines_safely(r, c)
285       #calculate the numbers for every cell
286       count_neighbours()
287       #start timer
288       tick()
289     if board[r][c]["mine"]:
290       #player clicked a mine, turning the square red and ending the game. (other mines are revealed with X's)
291         board[r][c]["revealed"] = True
292         buttons[r][c].config(text="", bg = "red", state = "disabled")
293         end_game(False)
294     else:
295       #Player clicked safe cell.
296       reveal_cell(r,c)
297       #Checks if player has won
298       check_win()
299   #What happens when you right click. (Flag mechanics)
300
301   def right_click(r, c):
302     global FlagLeft
303   #game already ended, do nothing
304     if GameOver: return
305       #can't flag a cell that has been uncovered already
306     if board[r][c]["revealed"]: return
307   #cell has already been flagged, so remove the flag and give player flag back
308     if board[r][c]["flagged"]:
309       board[r][c]["flagged"] = False
310       FlagLeft = FlagLeft + 1
```

```
    else:
312   #No flag is on the square, so place a flag and remove a flag from player.
313   if FlagLeft > 0:
314     board[r][c]["flagged"] = True
315     FlagLeft = FlagLeft - 1
316 #update flag counter and redraw the cell.
317 label_mines.config(text="Flags: " + str(FlagLeft))
318 draw_cell(r, c)
319
320 def check_win():
321 #Loop through every cell on the board
322 for r in range(ROWS):
323   for c in range(COLS):
324     #if we find any safe cell that hasn't been revealed yet, then the game isn't
    over, return, and keep playing.
325     if board[r][c]["mine"] == False and board[r][c]["revealed"] == False:
326       return
327       #if we check the entire board without returning, every safe cell has been
    revealed and the player wins!
328 end_game(True)
329
330 def end_game(won):
331 global GameOver
332 GameOver = True
333 if won:
334   #calculates how long the player took
335   TimeElapsed = int(time.time() - StartTime)
336   button_reset.config(text = "WIN")
337   label_msg.config(text = "YOU WON IN " + str(TimeElapsed) + " seconds!", fg =
    "white")
338 else:
339   button_reset.config(text = "LOSS")
340   label_msg.config(text = "BOOM! Close one! Try Again!", fg = "white")
341   #reveals the mines the player didn't find after losing.
342   for r in range(ROWS):
343     for c in range(COLS):
344       if board[r][c]["mine"] and not board[r][c]["revealed"]:
345         board[r][c]["revealed"] = True
346         draw_cell(r, c)
347         #disables every button so the player can't keep clicking after losing.
348   for r in range(ROWS):
349     for c in range(COLS):
350       buttons[r][c].config(state = "disabled")
351
352 def tick():
353 global timerID
354 #only update the timer if game is still going
355 if not GameOver and GameStarted:
356   elapsed = int(time.time() - StartTime)
357   label_timer.config(text = "Time: " + str(elapsed))
```

```
    #tells tkinter to call tick() again after 1 second or 1000 milliseconds. We update the timer every second without freezing our window
    timerID = root.after(1000, tick)

#This function places the mines
def place_mines_safely(first_r, first_c):
  currently_placed_mines = 0
  while currently_placed_mines < MINES:
    #Pick a random row and column on the board
    r = random.randint(0, ROWS - 1)
    c = random.randint(0, COLS - 1)

    #Skip if the cell was just clicked by the player for their first click. Making sure the first click is never a mine, as I mentioned before.
    if (r == first_r and c == first_c):
      continue
    if not board[r][c]["mine"]:
      board[r][c]["mine"] = True
      currently_placed_mines += 1

def new_game():
  global GameOver, GameStarted, FlagLeft, StartTime, timerID
#cancels the old timer if one is running so there isn't 2 timers going on at once
  if timerID:
    root.after_cancel(timerID)
    timerID = None
#Resets all the game variables back to their original, starting values.
  GameOver = False
  GameStarted = False
  FlagLeft = MINES
  StartTime = 0
#Reset all the labels and buttons back to their starting appearance
  button_reset.config(text = "  ")
  label_mines.config(text = "Flags: " + str(FlagLeft))
  label_timer.config(text="Time: 0")
  label_msg.config(text = "")
#Builds a fresh empty board. (no mines until first click)
  board_building()
  #remove all the old cell buttons from the board frame
  for widget in board_frame.winfo_children():
    widget.destroy()
  buttons.clear()
  #creates a new button for every cell and places it in the grid
  for r in range(ROWS):
    row_buttons = []
    for c in range(COLS):
      btn = tk.Button(
        board_frame, text = "", width = 2, height = 1, font = ("Arial", 14, "bold"), bg = "grey", relief = "raised")
      #grid() places the button at position (row, column) in the board frame
```

```
        btn.grid(row = r, column = c)
406         #Bind left click and right click with their functions
407         #Use lambda with r = r and c = c to remember which cell this button belongs to
408         btn.bind("<Button-1>", lambda e, r = r, c = c: left_click(r, c))
409         btn.bind("<Button-3>", lambda e, r = r, c = c: right_click(r, c))
410         row_buttons.append(btn)
411       buttons.append(row_buttons)
412
413   def start_game():
414     #Starts a new game and switch to the game screen
415     new_game()
416     show_screen(screen_game)
417   #shows the main menu first when the program starts.
418   show_screen(screen_main)
419   #mainloop() hands control to tkinter, keeping the window open, listening for clicks,
      etc.
420   #The program stays here until the window is closed
421   root.mainloop()
```